



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

MINI PROJECT REPORT

FACULTY OF COMPUTING

2024/2025 SEMESTER 2

SECJ2154

OBJECT ORIENTED PROGRAMMING

LECTURER'S NAME: DR. MUHAMMAD KHATIBSYARBINI

SECTION: 03

No	Name	Matric No
1.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112
2.	IMAN ABADI BIN MOHD NIZWAN	A23CS0084

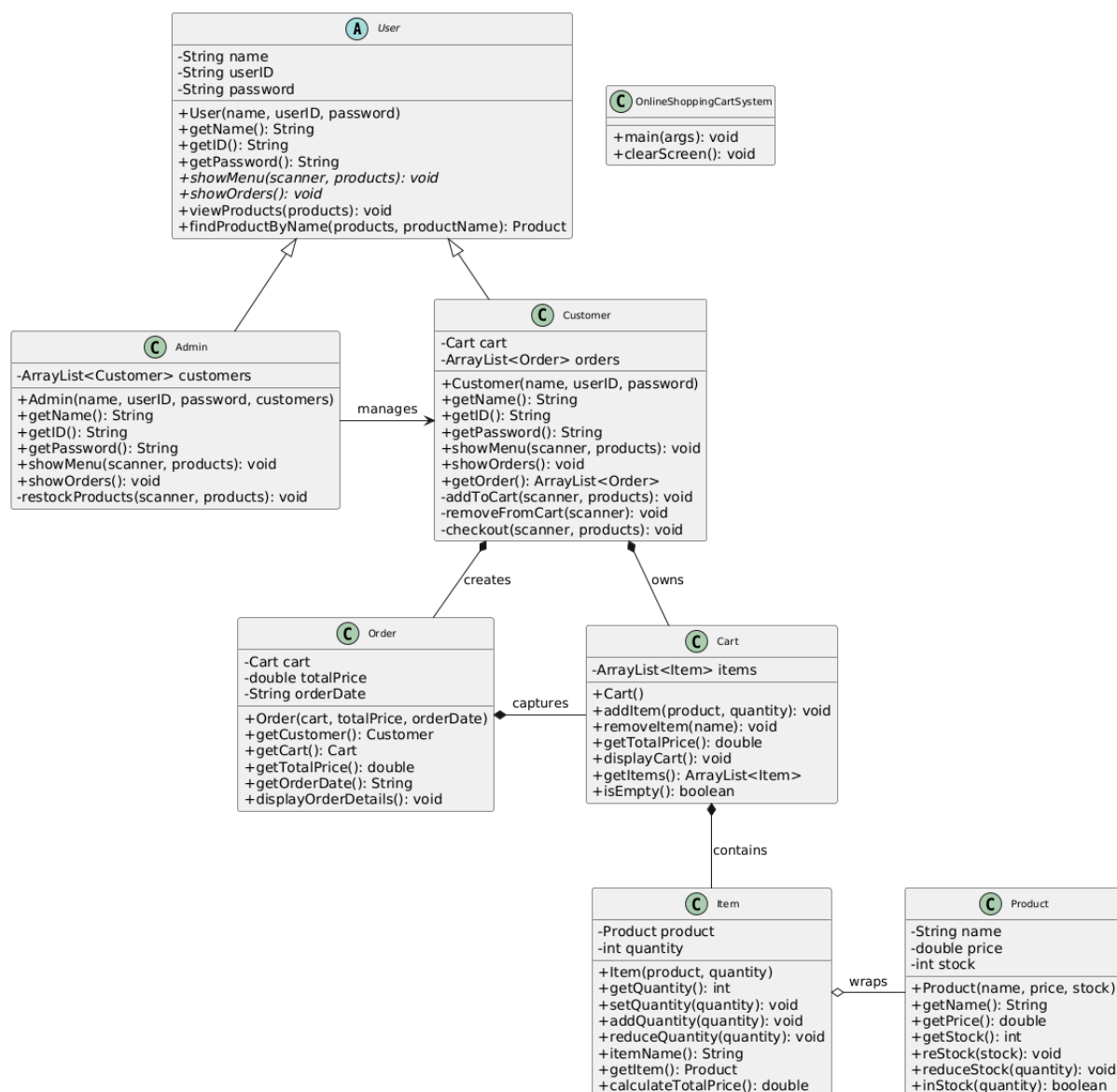
TABLE OF CONTENTS

PROJECT DESCRIPTION	3
UML DIAGRAM OF ONLINE SHOPPING CART SYSTEM	3
CLASSES EXPLANATION	4
IMPLEMENTATION OF KEY OOP CONCEPTS	5
a. ArrayList	5
b. Class Relationships	6
i) Association (General "has-a" relationship)	6
ii) Aggregation (Weak "has-a" relationship - part can exist independently)	6
iii) Composition (Strong "has-a" relationship - part cannot exist independently)	6
c. Inheritance	7
d. Polymorphism	7
e. Exception Handling	9
APPENDIX	10

PROJECT DESCRIPTION

This project is a simple online shopping cart system built using Java. It consists of two different types of users, which are customers and admins. For customers, they are allowed to browse products, add them to their cart, and manage their orders. For admins, they are allowed to look at a list of products, manage their stocks, and view orders placed by customers. This project applies the core principle of object-oriented programming that will be further explained in the next sections.

UML DIAGRAM OF ONLINE SHOPPING CART SYSTEM



CLASSES EXPLANATION

1. User

The User class is an abstract that represents individuals that have valid credentials, such as name, userID, and password, to access the system. The class only has two abstract methods, which are displaying the menu for a specified user and showing orders.

2. Customer

The Customer class inherits all the attributes of the User class that has methods such as adding items to carts, removing items from carts, checking out their orders, and viewing orders that were placed.

3. Admin

The Admin class inherits all attributes of the User class that has a method such as viewing all customers' orders and restocking products.

4. Product

The Product class represents the products that are available within the online shopping system. It holds the details of the products, such as the products names, prices, and available stock.

5. Item

The Item class represents an item that contains the product object and quantity of the product. It can set, add and reduce the quantity of the product and calculate the price of the item.

6. Cart

The Cart class represents a cart that holds an array list of items from the Item class, adding and removing specific items based on the quantity of the item. It also has a method that shows a list of items in the cart, calculating the total price of the items in the cart.

7. Order

The Order class represents an order that holds the record of a cart from the customer. The class contains methods like displayOrderDetails to display all the details from the checkout.

8. OnlineShoppingCartSystem

The OnlineShoppingCartSystem class is the main class that connects all the classes together and manages their interactions.

IMPLEMENTATION OF KEY OOP CONCEPTS

This section elaborates on how core Object-Oriented Programming (OOP) concepts are implemented throughout the Online Shopping Cart System.

a. ArrayList

The system makes extensive use of `java.util.ArrayList` for managing dynamic collections of objects. `ArrayList` is a resizable array implementation, making it highly suitable for scenarios where the number of elements (like users, products, or items in a cart) can change during runtime. No `java.util.Vector` implementations are found in the provided code.

Code Sections Highlighting `ArrayList` Usage:

```
public class OnlineShoppingCartSystem {
    public static void main(String[] args) {
        ArrayList<User> users = new ArrayList<>();
        ArrayList<Customer> customers = new ArrayList<>();
        ArrayList<Product> products = new ArrayList<>();

        // Populating these lists with initial data
        Customer cust1 = new Customer("MOHAMED ALIF FATHI", "ali", "ali123");
        Customer cust2 = new Customer("SITI NURHALIZA", "siti", "siti123");
        customers.add(cust1);
        customers.add(cust2);
        Admin admin = new Admin("IMAN ABADI", "admin", "admin123", customers);
        users.add(admin);
        users.add(cust1);
        users.add(cust2);

        products.add(new Product("Laptop", 6000.00, 10));
        products.add(new Product("Phone", 3000.00, 20));
        // ... more products
    }
}
```

At system startup, `ArrayList` is used to create and manage the master lists of Users, Customers, and Products.

b. Class Relationships

The system demonstrates various types of relationships between classes, which are fundamental to object-oriented design.

i) Association (General "has-a" relationship)

```
public class Admin extends User {  
    private ArrayList<Customer> customers; // Admin manages customers  
    public Admin(String name, String userID, String password, ArrayList<Customer> customers) {  
        super(name, userID, password);  
        this.customers = customers; // Customers are passed in, implying they exist independently  
    }  
    // ...  
}
```

Admin associated with Customer. An Admin manages a collection of Customer objects, but these Customer objects are created externally (in OnlineShoppingCartSystem) and can exist even if the Admin object is removed.

ii) Aggregation (Weak "has-a" relationship - part can exist independently)

```
public class Item {  
    private Product product; // Item makes a copy of a product  
    public Item(Product product, int quantity) {  
        this.product = product; // Item holds a specific Product  
        this.quantity = quantity;  
    }  
}
```

Item aggregates Product. An Item represents a specific Product along with a quantity.

iii) Composition (Strong "has-a" relationship - part cannot exist independently)

```
public class Customer extends User {  
    private Cart cart; // Cart is part of a Customer  
    public Customer(String name, String userID, String password) {  
        super(name, userID, password);  
        this.cart = new Cart(); // Cart is instantiated within the Customer's constructor  
    }  
}
```

Customer is composed of Cart. Each Customer object implicitly owns its Cart. If the Customer object is destroyed, its associated Cart object is also destroyed.

c. Inheritance

Inheritance is a core OOP principle that establishes an "is-a" relationship, promoting code reusability and a hierarchical structure.

```
public class Admin extends User {  
    // ... Admin specific fields and methods ...  
}
```

Admin inherits from User. An Admin "is a" User. It inherits common attributes like name, userID, password and basic behaviors defined in the User class, then adds its unique administrative functionalities.

```
public class Customer extends User {  
    // ... Customer specific fields and methods ...  
}
```

Customer inherits from User. Similarly, a Customer "is a" User. It also inherits the common User properties and behaviors, extending them with features relevant to a shopping customer.

d. Polymorphism

Polymorphism, meaning "many forms," allows objects of different classes to be treated as objects of a common type (their superclass), enabling flexible and adaptable code. This is primarily demonstrated through method overriding.

showMenu() and showOrders() Methods: The User abstract class declares showMenu() and showOrders() as abstract methods, requiring all concrete subclasses to provide their own implementation:

```
// In User.java  
public abstract void showMenu(Scanner scanner, ArrayList<Product> products);  
public abstract void showOrders();
```

Both Admin and Customer classes provide their specific implementations for these methods:

```
// In Admin.java  
public void showMenu(Scanner scanner, ArrayList<Product> products) {  
    // Admin-specific menu options and logic  
}  
public void showOrders() {  
    // Logic for Admin to view all customers' orders  
}
```

```
// In Customer.java
public void showMenu(Scanner scanner, ArrayList<Product> products) {
    // Customer-specific menu options and logic
}
public void showOrders() {
    // Logic for Customer to view only their own orders
}
```

In OnlineShoppingCartSystem.java, after successful login, the system treats the logged-in user as a generic User object, but the showMenu() method call dynamically invokes the correct implementation based on whether the currentUser is an Admin or a Customer:

```
// In OnlineShoppingCartSystem.java
User currentUser = null;
// ... (Logic to assign Admin or Customer object to currentUser) ...
currentUser.showMenu(scanner, products); // Polymorphic call: executes Admin's or Customer's showMenu
```

This demonstrates how the same method call can exhibit different behaviors depending on the actual runtime type of the object, a core aspect of polymorphism.

e. Exception Handling

The system incorporates try-catch blocks to manage potential runtime errors effectively, primarily focusing on `NumberFormatException` when parsing user input and general `Exception` for broader error conditions. This strategy prevents unexpected program termination and provides informative feedback to the user.

In `Admin.java` (within `showMenu` and `restockProducts` methods):

```
// Admin.java - showMenu example
public void showMenu(Scanner scanner, ArrayList<Product> products) {
    while (!logout) {
        try {
            // ... menu choice processing ...
        } catch (Exception e) { // Catches general exceptions for menu operations
            System.out.println("An error occurred: " + e.getMessage());
            System.out.println("Please try again.");
        }
    }
}

// Admin.java - restockProducts example
private void restockProducts(Scanner scanner, ArrayList<Product> products) {
    try {
        // ... input reading ...
        quantity = Integer.parseInt(quantityInput); // Potential NumberFormatException
        // ... restocking logic ...
    } catch (NumberFormatException e) {
        System.out.println("Invalid quantity. Please enter a valid number."); // Specific handling for bad numeric input
        return;
    } catch (Exception e) {
        System.out.println("Error restocking product: " + e.getMessage()); // General error for restock process
    }
}
```

This consistent use of try-catch blocks across critical user interaction points ensures the application's stability and provides a robust user experience by effectively managing potential errors.

APPENDIX

Personal Github Link: <https://github.com/AlifFathi/OOP-Project-Online-Shoping-Cart>

OOP Class Repository Link:

<https://github.com/Dr-Khatib/OOP-Section3-2425-2/tree/main/StudentFile/MOHAMED%20ALIF%20FATHI/MiniProject>

Video Presentation: <https://youtu.be/nLG1PzUcNT8>